26

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Mesaj Kuyrukları vs Olay Akışları: RabbitMQ vs Apache Kafka

Message Queues vs Event Streams: RabbitMQ vs Apache Kafka

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ RABBITMQ (Akıllı Broker, Aptal Tüketici - Smart Broker, Dumb Consumer) ]
Mesajlar kuyruğa gelir ──► Tüketici okur ve ACK atar ──► Mesaj kuyruktan SİLİNİR!
(İş kuyrukları, karmaşık yönlendirme - Topic/Header Exchange, RPC için idealdir)

[ APACHE KAFKA (Aptal Broker, Akıllı Tüketici - Dumb Broker, Smart Consumer) ]
Mesajlar sıralı bir kütüğe (Commit Log) yazılır ──► Asla silinmez (Günlerce durur!)
Tüketici kendi okuma noktasını (Offset) yönetir ──► İsteyen başa sarıp tekrar okur!
(Yüksek hacimli olay akışı, metrikler, Event Sourcing, saniyede milyonlarca QPS)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

RabbitMQ bir postane kuryesidir: Çantasından mektubu çıkarır, kapınızı çalar, elinize verir ve imzayı aldıktan sonra (ACK) o mektubu sistemden tamamen siler. Mektup bir daha geri gelmez. Apache Kafka ise gazete arşividir: Yazılan tüm haberler devasa ciltli bir deftere (Append-Only Commit Log) sırayla yapıştırılır ve ASLA yırtılıp atılmaz! Siz bir araştırmacı olarak defterin 50. sayfasına kadar okuyabilirsiniz (Offset); arkadaşınız ise 10. sayfada olabilir. Hatta isterseniz 1. sayfaya geri dönüp tüm yılı baştan sona tekrar okuyabilirsiniz (Replayability)!

🇬🇧 ENGLISH (ELI5)

RabbitMQ is a specialized courier: It pulls a letter from its satchel, rings your doorbell, hands it to you, collects your signature (ACK), and deletes the letter from the system forever. Apache Kafka is a bound newspaper archive: every article published is permanently glued onto an immutable, sequential commit log and NEVER discarded! You can read up to page 50 (Offset), while a colleague reads at page 10. You can even rewind your bookmark back to page 1 and reread an entire decade of history from scratch (Replayability)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) RabbitMQ (AMQP): Mesaj odaklıdır. Akıllı yönlendirme (Exchange tipleri: Direct, Fanout, Topic, Headers). Mesaj tüketici tarafından onaylandığında (ACK) broker'dan temizlenir. Düşük gecikmeli, karmaşık kurallı iş kuyrukları ve bildirimler için mükemmeldir; ancak geriye dönük yeniden okuma (Replay) desteklemez. 2) Apache Kafka: Dağıtık, append-only commit-log mimarisidir. Broker mesajı silmez; mesajlar retention süresi (ör. 7 gün) boyunca diskte sıralı tutulur. Tüketiciler kendi okuma indekslerini (`Offset`) yönetir. Sıfır Kopyalama (Zero-Copy OS sendfile) tekniği sayesinde saniyede milyonlarca olayı (GB/sn bant genişliği) kayıpsız işler. Event Sourcing, CQRS ve analitik akışlar için MAANG standardıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) RabbitMQ (AMQP): Message-oriented broker with sophisticated routing topologies (Exchange bindings: Direct, Topic, Fanout, Headers). Once acknowledged, messages are expunged from queue memory. Tailored for discrete background jobs, task queues, and low-latency notifications; lacks historical replayability. 2) Apache Kafka: Distributed, append-only, partitioned commit log. Messages persist immutably regardless of consumer state according to retention policies (e.g. 7 days or infinite compaction). Consumers advance localized offsets independently. Utilizing Linux kernel zero-copy (`sendfile` bypassing user-space memory buffers) and batch compression, Kafka handles millions of events/sec at multi-gigabyte throughput. De facto backbone for event-driven streaming and audit logging.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Kafka Offset Tabanlı Tüketim Modeli Simülasyonu
import java.util.*;

public class KafkaCommitLogSim {
    private final List<String> commitLog = new ArrayList<>();

    public synchronized long appendEvent(String event) {
        commitLog.add(event);
        return commitLog.size() - 1; // Üretilen Offset değeri
    }

    // Tüketici kendi offset'i ile istediği yerden okur (Replay yeteneği!)
    public synchronized List<String> pollEvents(long fromOffset, int batchSize) {
        List<String> results = new ArrayList<>();
        for (long i = fromOffset; i < commitLog.size() && results.size() < batchSize; i++) {
            results.add(commitLog.get((int) i));
        }
        return results;
    }
}

void main() {
    var kafka = new KafkaCommitLogSim();
    kafka.appendEvent("ORDER_CREATED_#1");
    kafka.appendEvent("ORDER_PAID_#1");
    kafka.appendEvent("ORDER_SHIPPED_#1");

    // Tüketici A 0. offset'ten okuyor:
    IO.println("Consumer A (Tüm Tarihçe): " + kafka.pollEvents(0, 10));
    // Tüketici B sadece 2. offset'ten sonrasını okuyor:
    IO.println("Consumer B (Sadece Kargo): " + kafka.pollEvents(2, 10));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Kafka'yı basit bir e-posta gönderme kuyruğu için kullanırım' demeyin! Küçük ölçekli, basit iş kuyruklarında Kafka'nın partition ve consumer group karmaşıklığı aşırı mühendisliktir (over-engineering); basit kuyruklar için RabbitMQ veya AWS SQS daha uygundur.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not recommend Kafka for trivial background email dispatch queues. Operating Kafka clusters incurs ZooKeeper/KRaft and partition management complexity. For simple task dispatching, propose RabbitMQ or AWS SQS.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- RabbitMQ mesajı işlendikten sonra siler; akıllı yönlendirme ve iş kuyrukları için idealdir.
- Kafka mesajları kütükte (log) saklar; geriye dönük tekrar oynatma (replayability) sunar.
- Zero-copy disk transferi sayesinde Kafka saniyede milyonlarca olayı minimum CPU ile taşır.

🎯 Key Takeaways for the Interview

- RabbitMQ expunges messages upon delivery; ideal for complex routing and job queues.
- Kafka stores immutable commit logs, allowing consumers to rewind and reprocess history.
- Zero-copy OS primitives allow Kafka to stream millions of events/sec at bare-metal speeds.